A Top–down Three–way Merge Algorithm For HTML/XML Documents

Anastasios G. Bakaoukas*, Nikolaos G. Bakaoukas†
* School of Science and Technology,
Computing & Immersive Technologies Department,
University of Northampton, Avenue Campus, St. Georges Avenue, Northampton NN2 6JB, UK
Email: Anastasios.Bakaoukas@northampton.ac.uk
† School of Engineering and Sciences,
Computer Science and Programming Department, Foundation College,
Mitropoleos St. & Mnisikleous Campus, Athens, 105 56, Greece
Email: fccscourse@gmail.com

***Abstract*—Collaborative work with the need to keep HTML/XML code up–to–date is now becoming vital particularly in the Web Development field. In order to fully support collaborative work and resolve collaborative work related problems the need has arisen for a solution(s) to the fundamental problem of how to edit in an automated way parallel copies of an original HTML/XML code document and subsequently merge the copies of that document into a single one. A number of corresponding algorithms have been used in the past in order to address the problem (such as: Diff3, XmlDiff, DeltaXML & 3DM) but HTML/XML code complexity issues have now called for an algorithm that is specifically designed for this type of files as opposed to general purpose algorithms used for specialised purposes.**

In this paper a new approach to algorithmic merge of HTML/XML code documents is presented based on the "Three–way Merging" approach and the node–per–node comparison between ordered trees. For the merging procedure a particular methodology is followed where only the two "Current Versions" are required for the generation of the new document, and no involvement of the "Original Document" is necessary, eliminating that way the "Idempotent" problem from which all merging algorithms using the original document in the merging procedure are suffering from, since has been proven that they are fundamentally not idempotent. In addition, the algorithmic approach presented here allows for the identification and treatment of all the conflicts that arise during a HTML/XML code merging procedure in an ordered and clearly specified manner.

***Index Terms*—Algorithms, Trees, Version Control, Document Management, HTML/XML Diffing, Edit Graph, File Comparison, Paths in Graphs.**

I. Introduction

The fundamental problem of computer supported collaborative work, when this involves the creation of a final version of a HTML/XML code document out of an original and a number of edited copies, is at its core primarily related with the algorithm used for reintegrating the edited copies into a single document containing the changes introduced [1].

The most basic of the merging scenarios requires the existence of an "Original Document" O (usually considered the last common version) and two "Current Versions" T_1 & T_2 that have been modified in respect to O. This is the so called "Three–way Merging" approach. Currently, the most successful algorithm for carrying out such kind of merging operations is considered to be the "Diff3" algorithm. As has been shown, though, the "Diff3" algorithm does not produce reliable results under all circumstances, and what is more, the algorithm does not fully possess the "Local Replicas" (distinct and well separated edited regions are merged without conflicts), "Idempotence" (the results of merging are fully stable except where edits conflict), "Stability" (similar inputs lead to similar outputs), and "Near–complete Success" (when the inputs have been changed in similar ways) properties [1].

IBM's Alphaworks team developed the merging and differentiating XML tool "XmlDiff", which compares two XML files and displays their differences. The basic idea behind the operation of this tool is that, after the initial automated merging has taken place, the users will then be able to further manually merge the two XML files by directly selecting what remains and what is deleted in various locations of the produced XML file [2].

Differences identification tools such as "DeltaXML" attempt to work within the structure of the XML documents so the need for manual adjustment after the automated procedure has provided the merged version of the input documents, is considerably minimised [3].

Tools that are heavily relying on the "Three–way Merging" approach, like the "3DM" tool handle the merging problem with a built–in version control engine. Despite of the existence of this extension to the core algorithm mechanism these tools do not fundamentally take advantage at all of the presence of the HTML/XML tags in the documents and are only based on their judgement upon the general structure of them [4]. Also, the conflicts handling methodology of the tools look rather subjective. As a result, when a conflict(s) arise between two updated versions of the same document the tools tend to decide about the action taken based only on the locally identified priority level(s). The most clearly identified operations tend to be the "Insert" and "Copy" operations for these tools, something

that very often is subsequently leading to the creation of non–optimal differential results [5]. For XML documents only, there are both commercial and research tools that apply the "Three–way Merging" method but in general nowadays is more or less an established fact that this approach is best suited, in most of the cases, for use with general XML documents [6].

Because it has been recognised that HTML/XML documents cannot be accurately compared and merged by simple line–by–line tools like "Diff3" (mainly because the tree structure can be compromised during both the attempt to identify the differences and the attempt to perform a semi–automatic merge) algorithms alternative to the traditional ones have been extensively used for the purpose. Algorithms like the "Zhang–Shasha" algorithm, analyses two trees to find their minimal edit distance. The "XyDiff" algorithm, finds a non–optimal edit distance between trees and incorporates a broader definition of edit operations. The "DaisyDiff" algorithm presents a solution capable of handling several types of merging. Finally, the "Lindholm" merge algorithm can be used directly on the target documents without the need to use an explicit edit script.

Despite the employment of a wide range of computational techniques in creating merging algorithms, relatively recent research in the area has shown that considered state–of–the–art merging algorithms like "Phoenix", X-Diff and XyDiff were capable of achieving efficacy in 26 of the 56 tested scenarios, in 30 of the 56 scenarios and failed in providing any precise matching results, respectively [7]. This result is serving to most graphically illustrate the fact that the area of merging algorithms is a very much open area for computational innovations.

The method presented in this paper for synchronising (merging) two HTML/XML "Current Versions" T_1 & T_2 that have emerged after two different sources have applied modifications to an "Archive Document" (alternatively also known as "The Last Common Version" or "Original Document"), is taking advantage of the ordered structure of a HTML/XML document and elaborates on a set of merging rules that emerge out of this property of ordered structure inherited in this type of documents. The algorithm focuses on merging the documents' HTML/XML programming code with an emphasis on the resultant information becoming available to the user after the output HTML/XML programming code produced by the merging procedure has been executed on a browser application, and in general is a straight forward and easy at its implementation algorithm that still manages to achieve efficient results. Because of its algorithmic structure being HTML/XML programming code oriented, the algorithm can be more accurately described as one that belongs to the class of algorithms in the area that can be used primarily as programming tools rather than taken as comparable with algorithms that focus in merging the information contained in a HTML/XML document structure.

In this respect, the algorithm can be used as a fast alternative of more sophisticated algorithms in the field, capable of producing reliable results, without the need of excessive computational power. In many cases the results produced by this simple algorithm are directly comparable with those by other far more complicated algorithms, something that makes it particularly attractive especially in cases where an algorithm's response time is of fundamental importance.

The material in this paper is presented in a total of six sections. In section I an overview of the operational procedure of some of the best known algorithms in the area is provided for the benefit of the reader, along with introductory concepts applicable in the field of three–way merging algorithms, as they are key elements in the algorithm presented here and an understanding of their relevance is mandatory for what is discussed in later sections. Section II presents and analytically discusses the "Tag" and "Nesting" characteristics of a structured HTML/XML document's programming code. Some algorithmic oriented classifications crucial for understanding the inner workings of the algorithm discussed in this paper are introduced and explained in section III. Section IV contains a detailed explanation of all the individual stages of the algorithm proposed, while section V presents experimental results that evaluate the level of success of the algorithm. Finally, section VI includes the conclusions of the paper.

II. The "Tag" & "Nesting" Characteristics of a HTML/XML Document

The Hypertext Mark–up Language is essentially the programming language for creating HTML/XML documents to view through a Browser application. The HTML/XML instructions exist as independent elements that are known as TAGs and each fully fledged HTML/XML document can theoretically consist of an n number of TAGs. These TAGs represent and describe the structure of the displayed HTML/XML document, meaning, the way it will be presented to the end user.

```
<!doctype html>

<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>The Basic HTML Page...
    </title>
  </head>
  <body>
    Hello World!
  </body>
</html>
```

Fig. 1: The fundamental constituent parts of every HTML/XML document. Purple colour is signifying a directive to the interpreter, Green colour is signifying a Tag, Red colour is signifying an attribute to a Tag, Blue colour is signifying a parameter to an attribute and Black colour is signifying plain text.

Concentrating only on the structural features of a HTML/XML document that are of interest to a synchronisation (merging) algorithm, a typically structured HTML/XML document consists of two fundamental sections (Fig. 1). That is, the "Head" and the "Body" sections, signified respectively

by the $< head >< /head >$ and $< body >< /body >$ pair of Opening and Closing tags. In their turn, both of these sections are encapsulated within a $< html >< /html >$ pair of tags that indicate the type of the document and acting as the "Starting" and "Ending" landmarks of the entire document.

Between the two fundamental sections of a HTML/XML document the one of particular interest, as far as merging algorithms are concerned, is the $< body >< /body >$ section. It is in this very section that the information people see in their browsers is contained. Naturally, the target of every merging algorithm ever developed is to achieve an as accurate as possible synchronisation of content in exactly this section.

```
<!DOCTYPE html>

<html>
  <body>
    <section>
      <h1>
        [Text 1]
      </h1>
      <p>
        [Text 2]
      </p>
    <section>
      <h2>
        [Text 3]
      </h2>
      <p>
        [Text 4]
      </p>
      <p>
        [Text 5]
      </p>
      <p>
        [Text 6]
      </p>
    </section>
    </section>
  </body>
</html>
```

Fig. 2: An example of a fully structured HTML/XML document.

The above provided description of the structure of a HTML/XML document reveals and a fundamental rule in structuring HTML/XML documents, that of "Nesting". This rule is a very significant one because adds a logical, mathematical in its nature, structural arrangement for every HTML/XML document by demanding that when we introduce an Opening Tag A and then another Opening Tag B, we have to successively introduce later first a Closing Tag for B and then a Closing Tag for A (Fig. 2).

In subsequent sections the fact that modern browsers are only very loosely follow the "Nesting" rule will be demonstrated, producing because of that a result with very minor differences when the rule is violated between dissimilar tags and an identical result for cases where the rule is violated between similar tags.

III. An Algorithm Oriented Classification of HTML/XML TAGs

In this section we will formulate a clear classification of HTML/XML Tags mainly according to two criteria, the second of which has not been previously introduced and used in any of the existing algorithms in the field. These are, a) the syntactical structure of the Tags and, b) a hierarchical order of Tags.

The categorisation of the HTML/XML Tags to "Paired" and "Unpaired" is well known and as a classification has been in use for many years. For the benefit of the reader, a HTML/XML Tag is considered as "Paired" when other code have to be placed between an Opening Tag and its corresponding Closing Tag as shown in the simple example below for the case of a text block:

```
<p> Some Text </p>
```

The Paragraph Tag is one of the most characteristic and well-known Tags and also one that belongs to the "Paired" category, since it requires an Opening Tag ($< p >$) and a Closing Tag ($< /p >$). The introduced between the Opening and Closing Tags block of text, is automatically formatted by the browser to the form dictated by the nature of the Tag. The same pattern is followed by all the Tags belonging to the same category.

In the "Unpaired" category of Tags (also known as "Singular" or "Stand–alone"), belong those Tags that simply require only an Opening part in their syntax and not a Closing part:

```
<br>

 or

<hr>
```

Both the Line Break and the Thematic Change Tags do not syntactically require the existence of a Closing Tag and no other block of code is expected to be found directly attached to them either. In the past a further break down of categories has been proposed (like, "Formatting" Tags, "Page Structure" Tags, "Control" Tags, etc.) but for the purposes of this discussion we will only consider these two fundamental ones. Further, for the algorithm we introduce in this paper extended forms of "Paired" and/or "Unpaired" Tags with attributes are treated in a rather binary way in the sense that they are considered being consisted of an Opening Tag and a Closing Tag only when the extension text has not been modified in the T_1 & T_2 documents, and as by an Opening Tag, a text part and a Closing Tag, when the extension text has been modified in either one of the two documents. For example, the following extended form of the Opening Tag for $< body >$ is considered as only representing one Tag, with the $style$ part ignored in the algorithm and considered merged with the main Tag label when no modifications have been applied, while it would have been considered as consisting out of two parts otherwise.

```
<body style = "font-size:18px;">
```

According to this assumption, the following lines of HTML/XML code are considered to be representing an Open-

ing Tag, a Text part and a Closing Tag in the algorithm, respectively (for the case of no modifications introduced in the *style* part, otherwise of two text parts):

```
<p style="color:#00b050;">
  First Paragraph
</p>
```

The text parts in the HTML/XML code structure are assigned by the algorithm, for convenience in handling them algorithmically, a unique label. The labelling takes place in a top–down fashion and has the following form:

```
[Text {Order of Appearence Number}]
```

Finally, for a better utilisation of the part of the algorithm that reads a HTML/XML document we propose a structural arrangement of the code close to those arrangement practises that practised in modern programming. In such practises a "New Line" is introduced for every new command and "Tabs" are heavy used to classify lines of commands as belonging in the same logical block of commands that collectively achieve a particular task. The role of Opening and Closing directives for logical blocks of commands (e.g. the "{" and "}" characters in C++) is naturally assigned to the Opening and Closing Tags in a HTML/XML document, so there is no need for the introduction of another scheme in this sector. This structural arrangement is directly equivalent to the *Indentations* way of representing a tree structure. A fully fledged example of the proposed structural methodology for a HTML/XML document to be used with the algorithm we propose here is presented below:

```
<!DOCTYPE html>

<html>
  <head>
    <title>
      Sample of HTML Code
    </title>
  </head>

  <body style = "font-size:18px;">
    <p style = "color:#00b050;">
      First Paragraph
    </p>

    <p>
      Second Paragraph
    </p>
  </body>
</html>
```

So, the parts of the HTML/XML code that actually need to be consider by the algorithm can be isolated as:

```
<html>
  <head>
    <title>
      [Text 1]
    </title>
  </head>

  <body>
    <p>
      [Text 2]
    </p>

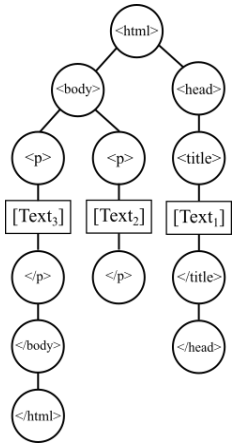
    <p>
      [Text 3]
    </p>
  </body>
</html>
```

IV. The Three–way HTML/XML Programming Code Merging Algorithm

In this section the merging algorithm is explained in its operational details and examples of its operation on HTML/XML documents' programming code are provided for the purpose of highlighting its main operational features. In general the new approach is based on the correspondence ratio between the branched arrangements the documents' programming code. More specifically, the merging procedure includes the following five processing stages:

Stage 1: The "Original Document" O and the two "Current Versions" T_1 & T_2 that contain modifications in respect to O are provided as input to the algorithm.

Stage 2: As is the case with all algorithms in the field, in the inner structure of the algorithm we present here every HTML/XML document's programming code is represented as a Node–labelled Ordered Tree, that is, a DOM (Document Object Model) tree, with each HTML/XML Tag corresponding to a node of that tree. Since the tree is ordered, the children nodes of every parent node are ordered as well and they can be uniquely identified by their path. An example of this practice is provided below:



At this stage, the programming code of document O and that of the two documents to be merged T_1 & T_2 is converted into a DOM tree representation.

Stage 3: The algorithm compares document's O DOM tree with each of those for documents T_1 & T_2 and marks the nature of the operation that caused every difference is identified in them, e.g. as the result of an "Insert" operation, as a result of a "Delete" operation, etc. There are five operations that can be used to modify a HTML/XML document's programming code and reflected in its DOM tree arrangement. These are the operations: Delete, Move, Copy, Update and Insert. In the algorithm's code structure this takes the form:

```
struct PerformedOperation
   {
   unsigned char Operation;
   } Delete,Move,Copy,Update,Insert;
```

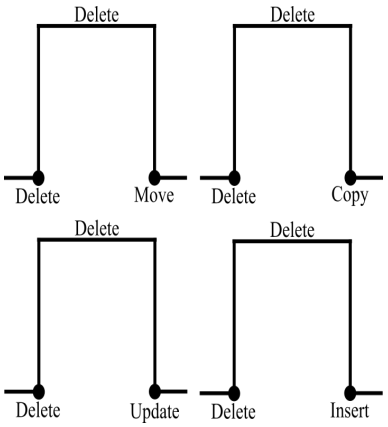
Stage 4: The algorithm, considering operations' priorities

and Tags' priorities, proceeds to merge the two documents' tree representations as per tree level, per level branch, per node. In other words, the algorithm seeks for an optimum way to map the two trees, on a node–to–node basis. The result of each merging instance will be the creation of a new node for the output tree.

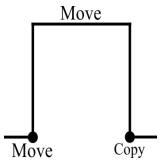
Since, achieving effective consolidation using a merging algorithm requires an effective method of handling conflicts the algorithmic approach allows for the identification and treatment of such conflicts through operations' and Tags' priorities.

The priorities are introduced so critical cases can be considered inside the algorithm. A typical example of such cases arises when an operation removes a sub–tree (Delete operation) while another concurrent one appends a node to this sub–tree (Update operation). Undeniably, this is a characteristic case of conflict. According to the operations' priorities that will be discuss below in details the optimum solution to this specific problem, so extensive experimentation indicates, is to assign a higher priority to the Delete operation so the sub–tree is removed even if the concurrent changes performed by the Update operation on this very sub–tree are lost. In general such a practice insures, at least in the grand majority of similar situations, data convergence. Some of the existing algorithms in the field include a kind of "Undo" function in order to avoid the loss of data by restoring through that function the lost changes. The algorithm presented here does not, out of choice, include such a functionality and the initiative for such an action is left with the user.

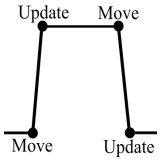
The operations' priorities are user specified but set initially to default values in the algorithm that have been decided on the basis of experimental results. The user can completely overwrite these default values introducing changes in order to achieve specific results (by having a specific pair of operations to result to another user specified operation). The default operations' priorities for the algorithm are as follows, with the bottom pair of operations representing input, while the top represents output after decision:



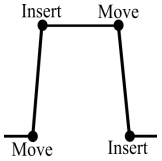
The above priorities arrangement indicates that the Delete operation by default possesses a higher priority than any of the other operations.



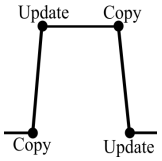
Since, in far more cases duplication of information in a HTML document has no meaning.



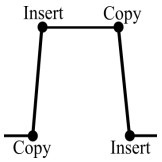
Since, maximum accuracy is achieved if we keep in a HTML/XML document the new content and then perform the Move with the new content included.



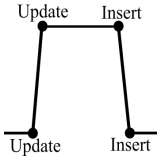
Since, maximum accuracy is achieved if we keep in the HTML/XML document the newly inserted line of code and then perform the Move on the old one.



Since, maximum accuracy is achieved if we first Update the existing line of code in the HTML/XML document and then Copy it to the new location.



Since, maximum accuracy is achieved if we first Insert the new line of code in the HTML/XML document and then Copy the existing one to the new location.

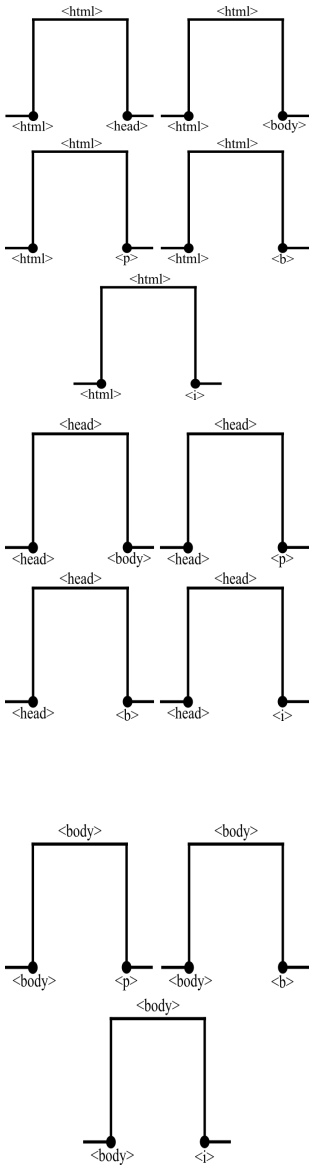


Since, maximum accuracy is achieved if we first Update the existing line of code in the HTML/XML document and then

Insert the new line at the new location.

After what presented so far must have now become apparent that the operations' priorities are functioning in the algorithm in a similar way to this of time–stamps in other algorithms in the field. The need for this arises from the fact that in a three–way merge situation we cannot attach any kind of a time–stamp to operations performed on a HTML/XML document's programming code in any meaningful way and at the same time we need to keep a logical order (free of contradicting acts) between those operations performed by the merging algorithm during the merging procedure.

As is the case with the operations' priorities, Tags' priorities are assigned default values that the user can completely overwrite introducing changes in order to achieve specific results. Again, a specific pair of Tags can be set to result to another user specified Tag, if this is what needed. The by default set of Tags' priorities for those Tags we are using later in the algorithmic examples are as follows:



** – <b>**: result **decision is taken according to the "Nested Parentheses" principle**.
** – <i>**: result **decision is taken according to the "Nested Parentheses" principle**.
<b> – <i>: result **decision is taken according to the "Nested Parentheses" principle**.
<Opening Tag> – </Closing Tag>: result **decision is taken according to the "Nested Parentheses" principle**.

The "Nested Parenthesis" principle resembles the one used in mathematics to introduce order of execution between calculations in an unambiguous way. The only difference, when used in the ordered trees sector, is that the role of Opening and Closing parentheses is taken over by Opening and Closing Tags. The following is an illustration of how the "Nested Parenthesis" principle is used in the logical structure of the algorithm to maintain a strictly hierarchical ordering between tree branches.

<html> <head> <title> </title> </head>...
((( ))...
...<body>   </body> </html>
...(( ) ( )))

The employment of the "Nested Parenthesis" principle, safe guards the algorithm against permitting for unorthodox matching between Tags, something that has been criticised as being one of the major drawbacks for many of the existing algorithms ("Diff3","XyDiff", "DaisyDiff", etc.).

Since, all priorities set in the algorithm are fully customisable and extendible, the user can introduce similar priorities for the full extend of HTML/XML Tags, something that covers completely the case of dynamic Tags very commonly found in XML code. Despite the above stated priorities, in the event of a conflict still arising the algorithm treats the situation in one of three ways. Namely, either deciding for an action by employing the "Nested Parentheses" principle on the Tags considered, or automatically selecting an action on the Tags, or marking it as to be handled later by the user in more complicated situations.

Finally, the only priority rules we need to set for the nodes in the ordered tree containing a text are: (a) all Opening Tags' nodes possess a higher priority than text nodes, (b) all Closing Tags' nodes possess a lower priority than text nodes, (c) nodes containing non-similar text that need to be merged are arranged at the output tree in the priority order dictated by their original locations in their corresponding, to be merged, documents T_1 & T_2 and, (d) "text duplication is not permitted"; that is, whenever two nodes from the modified versions of the original document are to be compared and these nodes contain the same text, this text is only once forwarded to the output document.

Stage 5: The output programming code DOM tree representation is provided, out of which the resulting document is created.

To conclude the discussion about the algorithmic principles of the method and for direct comparison reasons, the operational diagrammatic form of the algorithm is presented in Fig. 3 along with the equivalent for the "Diff3" algorithm.

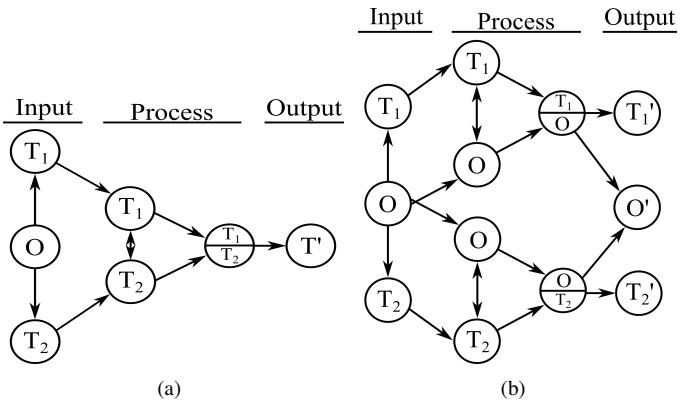
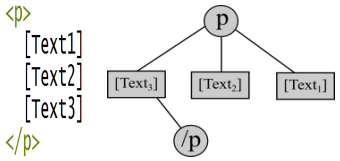


Fig. 3: (a) The operational diagram for the algorithm presented in this paper, (b) The operational diagram for the "Diff3" algorithm.

V. Experimental Results And Algorithm Evaluation

This section presents experimental results and attempts an evaluation of our algorithmic approach to the problem of merging two HTML/XML documents. The attempt here is not to provide a full list of results considering every possible situation that can be encountered in HTML/XML documents, but rather to present representative examples that illustrate the operational characteristics of the algorithm. For the rest of this section we will follow the "simple first, towards more complicated examples later" approach in how we proceed with the discussion on example cases. We first consider the following simple example HTML document and its corresponding DOM tree:



and then the modified documents T_1 & T_2, based on this original O, that are as shown in Fig. 4. In this case, despite the simplicity of the HTML code, the algorithms that do not incorporate the "Nested Parentheses" principle are bound to return at the output a wrongly arranged result, which experimentation with them suggests is the following:

```
<p>
  <b>
    [Text1]
    <i>
      [Text2]
  </b>
[Text3]
    </i>
</p>
```

The logic behind the merging algorithm that can produce such a merged document is shown in Fig. 5. Despite the fact that the wrongly arranged merged document, if observed through a browser, will produce acceptable display results, is still clearly structured wrongly in its arrangement.

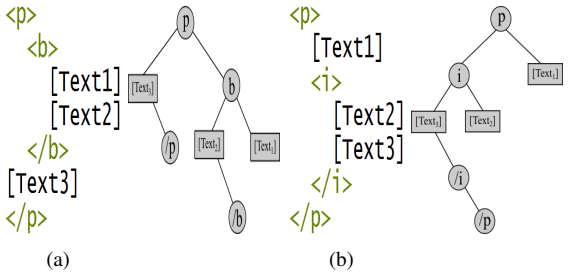


Fig. 4: (a) The modified document T_1, and (b) the modified document T_2.

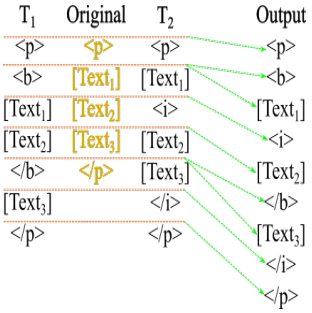


Fig. 5: The algorithmic logic diagram for resulting to a wrongly arranged structure for the merged document.

Because of having the "Nested Parentheses" principle incorporated into its algorithmic structure the algorithm proposed here is capable of recognising the situation and producing a result with the correct arrangement for the merged document:

```
<p>
  <b>
    [Text1]
    <i>
      [Text2]
    </i>
  </b>
  <i>
    [Text3]
  </i>
</p>
```

The logic behind the operation of the algorithm proposed, in this particular instance of a document, is as shown in Fig. 6.

Proceeding to the most complicated instance of an example, the original document O and its corresponding DOM tree are shown in Fig. 7.

The modified document T_1 and its corresponding DOM tree are shown in Fig. 8. In the DOM tree arrangement for this document, for ease of comparison, with green are marked the operations that have been performed on the original during the first modification stage.

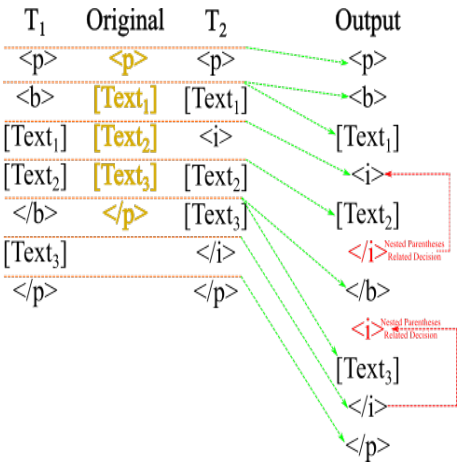
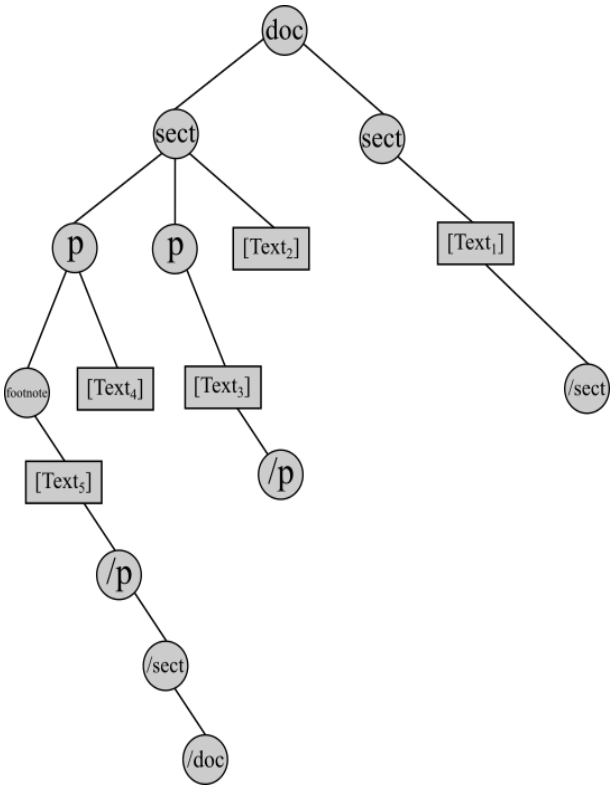


Fig. 6: The algorithmic logic diagram for resulting to a wrongly arranged structure for the merged document.

```
<doc>
  <sect title="Jokse" />
  <sect title="Student joke">
    <p>
      Q: How many students does it take to change a light bulb?
    </p>
    <p>
      A: None. Light bulb changing isn't part of the course.
      <footnote text="Except for projector bulbs" >
    </p>
  </sect>
</doc>
```

(a)

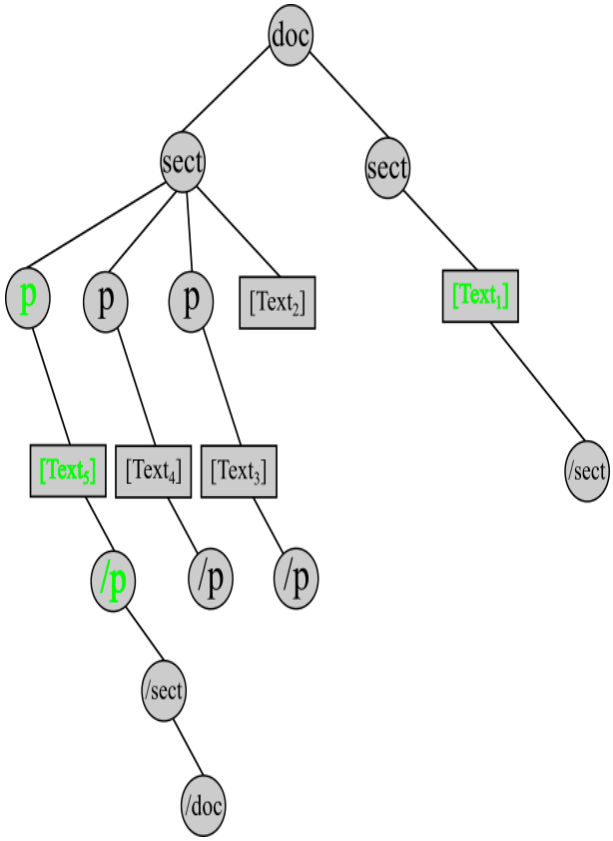


(b)

Fig. 7: (a) The original document O, and (b) its corresponding DOM tree arrangement.

```
<doc>
  <sect title="Jokes" />
  <sect title="Student joke">
    <p>
      Q: How many students does it take to change a light bulb?
    </p>
    <p>
      A: None. Light bulb changing isn't part of the course.
    </p>
    <p>
      A2: ''Will this be on the test?''
    </p>
  </sect>
</doc>
```

(a)



(b)

Fig. 8: (a) The modified document T_1, and (b) its corresponding DOM tree arrangement.

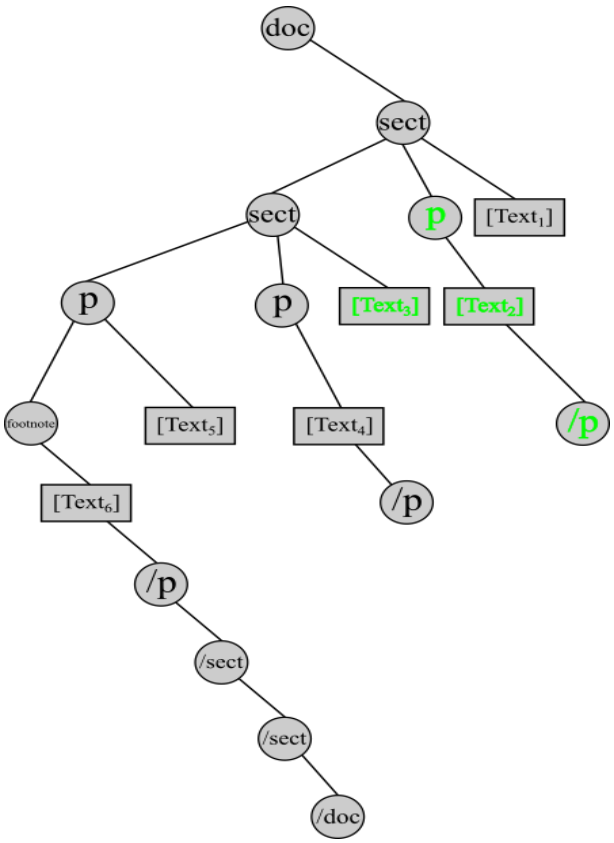
The modified document T_2 and its corresponding DOM tree are shown in Fig. 9. In the DOM tree arrangement for this document, again, with green are marked all the operations that have been performed on the original during the second modification stage.

As becomes apparent from observing fig. 8 & Fig. 9, the modifications introduced produce a far more complicated situation this time, with them extending from the text fields of the original document's structure to the extension part of some of the Tags. The algorithm's final merging output is as presented in Fig. 10.

Finally, the logic behind the operation of the algorithm proposed, in this particular instance of a more complicated document, is as shown in Fig. 11. Tags marked with red in

```
<doc>
  <sect title="Jokse">
    <p>
      Here are several good jokes
    </p>
    <sect title="Joke 1: Student joke">
      <p>
        Q: How many students does it take to change a light bulb?
      </p>
      <p>
        A: None. Light bulb changing isn't part of the course.
        <footnote text="Except for projector bulbs">
      </p>
    </sect>
  </sect>
</doc>
```

(a)



(b)

Fig. 9: (a) The modified document T_2, and (b) its corresponding DOM tree arrangement.

the second modified document's T_2 programming code signify Tags that are ignored by the code during the final stages of the merging procedure because with all the earlier Tags matched and completed according to the "Nested Parentheses" principle they correspond to Tags that produce no equivalent ones at the output document.

At this point we need to highlight the fact that not only the merged document is mapped on a correctly structured document's programming code, but also that as becomes clear from an observation of Fig. 11, the merging approach materialised by the algorithm is at the same time both a simple and straight forward as well as an effective one capable of producing descent results in a variety of different merging situations.

```
<doc>
  <sect title="Jokes">
  </sect>
    <p>
      Here are several good jokes
    </p>
    <sect title="Joke 1: Student joke">
      <p>
        Q: How many students does it take to change a light bulb?
      </p>
      <p>
        A: None. Light bulb changing isn't part of the course.
      </p>
      <footnote text="Except for projector bulbs">
      <p>
        A2:''Will this be on the test?''
      </p>
  </sect>
</doc>
```

Fig. 10: The merging output of the algorithm proposed in this paper for the second, more complicated instance of original document O.

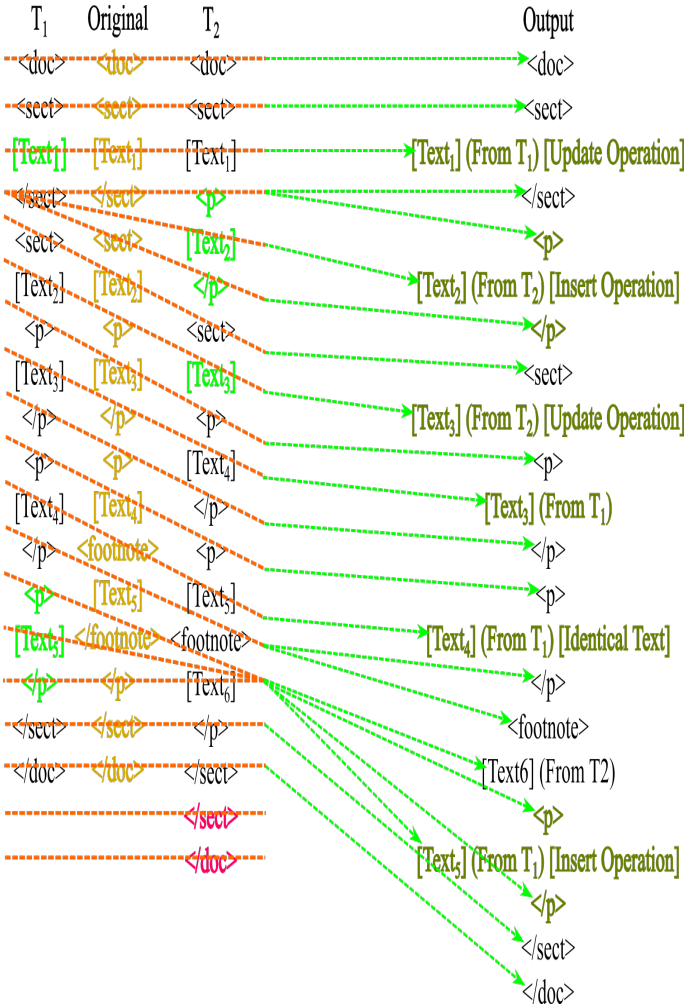


Fig. 11: The logic behind the algorithm's decision in merging the second example document presented in structural diagrammatic form.

VI. Conclusion

One of the major problems restricting the optimum operation of many of the already proposed three–way merging algorithms For HTML/XML documents is their strong dependency, when it comes to decision taking, on the original document O. In this paper, an optimum algorithm is proposed for the three–way merging approach to modified HTML/XML documents that completely eliminates the need for the original document O to be involved in the merging procedure. The proposed algorithm generates a merged document as its output that is encapsulating a well presented at the browser document within a properly structured HTML/XML programming code by gathering all important information for each of the Tags directly from the modified input documents. This is leading to an improvement algorithm over the existing algorithms in its category that can be used primarily (but not restricted to) as a programming assisting tool.

A key aspect of the algorithm presented in this paper is also its level of flexibility, which is provided by allowing configurable operations' and Tags' priorities, along with the "Nested Parentheses" principle. By doing so, the algorithmic structure provides the user with the ability to focus on either the HTML elements and/or the XML element features that finds more suitable according to the nature of the documents about to be merged.

Acknowledgment

The authors would like to thank editors and anonymous reviewers for their valuable and constructive suggestions on this paper.

References

[1] Sanjeev Khanna, Keshav Kunal & Benjamin C. Pierce, "A Formal Investigation of Diff3", In Arvind & Prasad, Editors, Foundations of Software Technology and Theoretical Computer Science (FSTTCS), December 2007.

[2] IBM Alphaworks, "XML Diff And Merge Tool Home Page", http://www.alphaworks.ibm.com/tech/xmldiffmerge.

[3] The "DeltaXML" Project, http://www.deltaxml.com [Accessed: 29 March 2019].

[4] Tancred Lindholm, "A Three–way Merge For XML Documents", Proceedings of The 2004 ACM Symposium on Document Engineering, 1-10, 10.1145/1030397.1030399, 2004.

[5] Hung Dinh, "A New Approach To Merging Structured XML Files", International Journal of Advanced Research In Computer Engineering & Technology (IJARCET), Volume 4, Issue 5, May 2015.

[6] Mouhamadou Lamine Ba, Talel Abdessalem & Pierre Senellart, "Merging Uncertain Multi–Version XML Documents", January 2013.

[7] Alessandreia Oliveira, Gabriel Tessarolli, Gleiph Ghiotto, Bruno Pinto, Fernando Campello, Matheus Marques, Carlos Oliveira, Igor Rodrigues, Marcos Kalinowski, Uverton Souza, Leonardo Murta & Vanessa Braganholo, "An Efficient Similarity–based Approach For Comparing XML Documents", Information Systems, Elsevier, Vol. 78, Pages 40–57, November 2018.

[8] "Document Object Model (DOM) Level 2 Core Specification v1.0", W3C Recommendation, http://www.w3.org/TR/DOM-Level-2-Core/Overview.html.

[9] Matthijs N, "HTML, The Foundation of The Web", Available Online At: http://www.wpdfd.com/issues/86/html_the_foundation_of_the_web/.

[10] Rozinajov V & Hluch O, "One Approach To HTML Wrappers Creation: Using Document Object Model Tree", In Proc. CompSysTech, 2009, pp.41–41.

[11] David Barnard, "Tree–to–tree Correction For Document Trees", http://citeseer.ist.psu.edu/47676.html.

[12] Grgory Cobna, "A Comparative Study For XML Change Detection, http://citeseer.ist.psu.edu/696350.html.

[13] Chawathe S. S., Rajaraman A., Garcia–Molina H. & Widom J.,"Change Detection In Hierarchically Structured Information", In Proc. of The 1996 ACM SIGMOD International Confernce on Management of Data, p.p. 493–504, Montreal, Canada, 1996.

[14] Cobena G., Abiteboul S. & Marian A., "Detecting Changes In XML Documents", In Proc. of The 18th International Conference on Data Engineering, p.p. 41–52, San Jose, CA, 2002.